Project 2 Brief

Numerical Solution of Elliptic Boundary Value Problems

Numerical Analysis of ODEs and PDEs · Weeks 5–9 · Team of 5 · English

---

1. Task

Solve an elliptic boundary value problem (usually in two dimensions) with a real-world background (see the Problem Pack, `problem_pack`, for the available directions). Discretise it, solve it, and — the core of the project — **verify that the numbers you report are supported by reproducible evidence**.

You are not asked to reproduce a textbook example. You are asked to **act like a consultant**: given a phenomenon (a steady temperature field, a membrane under load, a price surface, a micro-device near its critical voltage), produce a reliable, validated numerical answer, and explain — *quantitatively* — why your methods can be trusted.

2. Examples of work

Every item below is checked against a reference **you construct yourself** — we deliberately do not hand you benchmark numbers, because learning to judge "is this right?" is part of the project.

1. **Discretisation method: FDM or FEM (choose one)**
   - **FDM path**: the five-point stencil for $-\Delta$ (or the divergence form $-\nabla \cdot (a\nabla\cdot)$), assembled sparsely with `scipy.sparse`
   - **FEM path**: use the 1D derivation scaffold for training, then the minimal 2D pathway `fem2d_scikitfem_skeleton.py` (or an equivalent 2D library solve) with the mesh, weak form, finite-element space, quadrature, Dirichlet elimination, natural/Robin terms, and named L^2/energy errors made explicit
   - **Either path**: boundary treatment derived and tested (Dirichlet rows moved to the right-hand side; any Neumann/Robin closure stated with its order); a named-norm refinement protocol with $h, h/2, h/4, h/8$, fitted observed order, and the algebraic residual and tolerance reported at each level
2. **Justified solver choice**
   - Sparse direct (`spsolve`) or an iterative method (Jacobi / Gauss–Seidel / SOR) — the choice justified by the matrix structure and problem scale
   - Stopping criterion stated (residual, not iterate difference) and, for an iterative method, a tolerance demonstrably below the discretisation error
   - On one representative mesh, separate algebraic from discretisation error. With an iterative solver, repeat the solve at successively tighter tolerances until the reported discretisation error and the main quantity of interest stop changing. `spsolve` has no tolerance parameter: instead report its backward (residual) error and bound the forward error through a conditioning estimate, or cross-check the solve independently at a very tight tolerance. A small residual alone is not this demonstration in either case.
3. **One depth comparison** (★★; see the Problem Pack — Direction 2's two required closures count toward it)

- Two boundary closures, direct vs. iterative under matched tolerances, FDM vs. FEM, or smooth vs. nonsmooth data — the comparison is the content, with matched settings and a stated cost measure

4. **Report** ($\geq$ 8 pages, LaTeX; submit the PDF)
   - Structure: Background → Model → Discretisation → Verification → Numerical experiments → Conclusions (see the Project 2 report template: `06_Templates/latex_report_template_bvp.tex`)
5. **Code**
   - A zip archive with a README explaining how to run it
   - Everyone runs `run_all.py` to reproduce all figures
6. **Slides** (PDF, one set for your presentation wave)
   - The deck tells the full story arc (background → model → discretisation → verification → conclusions); each slide has a takeaway, not just bullet points
   - 10 min presentation + 3 min Q&A

Deadline: Monday of Week 10, 23:59. Late policy: 8-hour grace period, then -10% per hour (see the Syllabus).

How to verify without a given answer. There is no handed-out benchmark value in this course. Build your own ground truth with at least **two** independent checks:

1. **Analytic bank:** the project's `test_problem_bank.py` supplies exact solutions for the rectangular core problems (the disk direction has no bank entry — building its mask and boundary rule is part of that direction), each with a self-checking forcing term. The bank's self-check validates the analytic data; it does not test your solver — verify your implementation against the exact solutions in your refinement tables.
2. **Manufactured solutions:** choose your own u_*, apply your stated differential operator to it — $f = -\Delta u_*$ for the constant-coefficient directions, $f = -\nabla \cdot (a \nabla u_*)$ for the variable-diffusion direction — compute the boundary data, solve, compare. (For the MEMS nonlinear direction, derive the exact branch $\lambda(a)$ from the first integral instead — see the Problem Pack.)
3. **Independent oracle:** a second, differently obtained solution — `scipy.integrate.solve_bvp` for a 1D problem, a second discretisation (FDM vs. FEM), or a second solver at a much tighter tolerance. Agreement between two methods that share neither derivation nor implementation assumptions is the strongest check.
4. **Self-consistency:** two of your own methods with different structures must agree in the fine-mesh limit; the convergence-order plot itself is a check (a slope matching theory on a known case gives confidence).

In the report, state *which* checks you used, *what* tolerance/convergence criterion you accepted, and *where* your numbers come from. That chain of evidence is what we grade — not the numbers themselves.

3. Tiered tasks

Marked with stars. ★ = basic (must complete), ★★ = advanced (aim to complete), ★★★ = challenge (for strong teams). Stars classify scope: the higher levels open up the depth-and-novelty

space scored in Dimension B. Within a chosen scope, quality and verification decide the grade — excellent, thoroughly verified ★ work earns a strong mark.

- ★ Formulate the BVP; implement your discretisation — the five-point stencil (FDM) or the weak-form assembly (FEM) — and use a justified sparse solver.
- ★ Verify convergence on multiple meshes in a stated norm; explain the boundary treatment.
- ★★ One depth comparison (closures, solvers, FDM vs. FEM, or smooth vs. nonsmooth data; Direction 2's two required closures count).
- ★★ Cost versus accuracy: state the cost measure and report what you measured.
- ★★★ A singular domain, adaptive refinement, pure-Neumann nullspace treatment, a nonlinear direction (MEMS pull-in), or another original extension.

4. Suggested directions (details in the Problem Pack)

#	Direction	Core numerical lesson
1	Manufactured Poisson problem	MMS, order verification, error separation
2	Mixed boundary data	boundary-closure order, ghost points
3	Variable diffusion	discrete symmetry, flux continuity
—	Depth comparison	solver/boundary/method choice as evidence
⋆⋆⋆	MEMS pull-in (nonlinear)	Newton, continuation, "converged" is ambiguous

You may propose your own direction — state it in your Week-5 milestone entry.

5. Timeline and milestones

Week	Milestone	Deliverable due
5	Direction chosen, BVP stack installed, baseline test-problem run	problem definition sheet (Part 1; + Part 2 for FDM teams)
6	Solver working, first MMS refinement table, depth comparison chosen	first MMS refinement table (attached to the Weekly-Milestone entry)
7	**Verification clinic (Seminar 3)** — team pitch, formative	clinic readiness sheet (attached to the Weekly-Milestone entry)
8	**Voluntary first-wave presentation (Seminar 4)**	slides (PDF) + work-in-progress draft
9	**Second-wave presentation (Seminar 5)**	slides (PDF) at the seminar; report PDF (AI log + ICS) + code at the common deadline (Monday of Week 10, 23:59)

Why present in the first wave? It is voluntary but strongly encouraged: you present the work-in-progress, get the teacher's and peers' feedback *now*, catch modelling or implementation mistakes while they are still cheap to fix, and have a full week to improve before the final report. The presentation is assessed in the wave in which you present (each team presents once), and presenting work-in-progress is expected and acceptable: the report and code remain revisable until the common deadline, so a first-wave presentation does not lock in your final grade.

6. Grading

Project grades come from the course-level rubric (dimensions A–F), used for the team's single presentation wave, together with the report and code:

Dimension	Weight
A Mathematical Soundness	15%
B Depth and Novelty (topic, theory, algorithm)	15%
C Implementation (robust, reproducible)	15%
D Experiments & visualization	15%
E Writing & presentation	15%
F Individual Contribution (incl. ICS)	25%

Detailed descriptors: the course-level rubric (`../Numerical_PDEs_Assessment_Rubrics.pdf`).

7. What we will not accept

- A report that describes methods but shows no experiments
- Code that cannot be run by the instructor on a machine with dependencies installed
- A convergence claim without a stated norm and a refinement table
- A single team member who cannot explain the project (checked by cross-questioning in the Seminar Q&A)

Scored criterion (Dimension C, Implementation): the **core method must be correctly applied** — you may implement it directly or call a well-justified library routine (`numpy`/`scipy` are fine, including `scipy.sparse` and `scipy.integrate.solve_bvp`), but the choice of what to implement yourself vs. call must be **explicit and justified**, and you must be able to explain and modify the code. Vectorise where appropriate (matrix operations, not `for` loops over grid points). Correct application and justified choices — not re-implementing every scheme by hand — are what this dimension rewards.

Ask yourself, after every experiment: "If I were the instructor, would this convince me?" If the answer is no, keep working.

8. Using AI effectively

AI is welcome in this course, but it changes *what* you must do, not *whether* you can succeed. Two habits make the difference between learning and copying:

1. **Predict first, then let AI help step by step.** Before each new solve, write your predictions (pre-computation worksheet): signs, symmetry, expected order, limiting cases. Then use AI for individual steps, and verify each step against your predictions. If the AI's output contradicts a prediction, decide which is wrong — and why — before proceeding.
2. **Audit the output.** Any AI-generated solver code must pass the five-gate audit (transcription, structure and indexing, linearisation, independent reproduction, interrogation) in the AI-audit worksheet before its output is used. The audit record attaches to the AI Transparency Log.

The AI Transparency Log (appendix of the report template) records what you changed in AI output and why — that log *is* your defence, not your confession.